

Sec Corp Panel

Recommended Architecture and Approved Decision Register

Status Approved and implemented through Phase 7
Technology Angular 21 standalone Signals-first AG Grid
Document version 1.0 27 July 2026

Architecture position

Build only the dockable panel application. The existing enterprise shell remains responsible for window chrome, layout, docking, sizing, and the manual refresh command. The panel owns financial data retrieval, editing, calculation preview, optimistic concurrency, and grid rendering.

This document consolidates the recommended architecture, the implemented design, and all ten approved review decisions. It is intended to be included directly in a technical design or governance package.

Contents

1. Executive architecture recommendation 3

2. System context and component architecture 4

3. Feature-first folder structure 5

4. Domain model and view-model mapping 6

5. Signals state design and editing workflow 7

6. AG Grid implementation plan 8

7. REST JSON contract and optimistic concurrency 9

8. Sec Corp source mapping and representative data 10

9. Approved decision register - ten review questions 11

10. Testing and quality strategy 12

11. Delivery phases and implementation status 13

12. Runtime configuration and operational behavior 14

Appendix A. Architecture references 15

1. Executive architecture recommendation

Use a feature-first Angular 21 panel application with a reusable funding-panel core. Sec Corp is the first panel configuration, not a one-off grid. The design separates business data, exact decimal calculations, application state, grid view models, and HTTP DTO validation so additional funding panels can reuse the infrastructure safely.

Primary recommendation

Treat the server report as authoritative, keep unsaved edits in a sparse client overlay, recalculate valid previews immediately, and persist the complete editable snapshot state with version-based optimistic concurrency.

Goals

- **Production ownership:** predictable state transitions, runtime validation, clear failure recovery, and long-term reuse.
- **Financial correctness:** canonical decimal strings and decimal arithmetic, with no JavaScript floating-point calculations for money.
- **Safe collaboration:** versioned full-state PUT prevents stale users from overwriting newer database state.
- **Screenshot fidelity:** compact AG Grid presentation with dark rows, teal section bands, accounting negatives, and the approved row hierarchy.
- **Shell compatibility:** no duplicate title bar or layout system; the shell remains the host boundary.

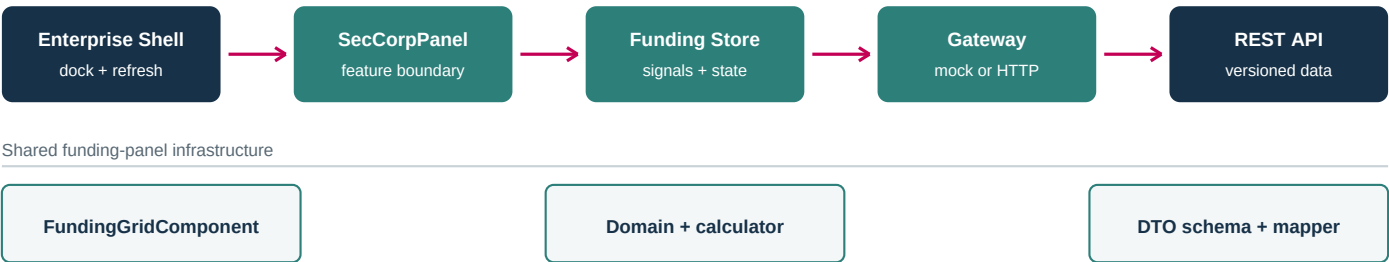
Technology constraints

Area	Approved approach
Framework	Angular 21, standalone components only, strict TypeScript, no any.
Reactivity	Signals-first with computed signals; effects only at imperative host and AG Grid boundaries.
Templates	Angular built-in control flow and prefer inject for dependencies.
Grid	AG Grid with runtime Enterprise license injection and explicit feature modules.
Quality	ESLint, Prettier, unit tests for business logic, and component tests for rendering and interaction.
Change detection	OnPush components in a zoneless Angular application.

Out of scope

- Shell docking, resizing, window controls, and title chrome.
- Automatic polling or background refresh.
- Authentication implementation; the shell and API gateway establish the session.
- Automatic client-side conflict merge.

2. System context and component architecture



Runtime responsibilities

Layer	Responsibility	Does not own
Enterprise shell	Docking, dimensions, panel lifecycle, business date, refresh command.	Financial state or cell editing.
SecCorpPanel	Feature composition, runtime configuration, status and conflict actions.	Grid internals or calculation rules.
FundingPanelStore	Load/edit/save state machine, preview report, dirty state, conflict and retry.	HTTP syntax or DOM mutation.
FundingGrid	Column definitions, financial formatting, editor lifecycle, accessible grid metadata.	Domain persistence.
Gateway + mapper	GET/PUT transport, DTO validation, domain mapping, conflict translation.	Presentation flags.
REST service	Authorization, atomic version compare, persistence, version increment, authoritative totals.	Panel rendering.

Component tree

```
AppComponent
  |-- RouterOutlet
    |-- SecCorpPanelComponent [feature-scoped providers]
      |-- FundingGridComponent [shared funding grid]
      |   |-- FundingAmountCellEditor [snapshot input + validation]
      |-- loading / empty / load-error state
      |-- saving / save-error / conflict action overlay

Enterprise shell (outside Angular panel)
  |-- PanelHostAdapter
    |-- refreshRevision signal [manual refresh command]
    |-- published PanelHostState [dirty, save status, canRefresh]
```

Key ownership rule

Each docked panel instance provides its own FundingPanelStore. Multiple Sec Corp panels therefore cannot leak edits, save status, conflicts, or refresh state into one another. Shared services remain stateless or runtime-configured.

3. Feature-first folder structure

```
src/app/
|-- core/
|   |-- config/           runtime configuration and validation
|   |-- grid/            AG Grid module and license setup
|   |-- host/            enterprise shell adapter
|-- features/funding-panel/
|   |-- domain/
|   |   |-- funding-report.ts    domain model and period identifiers
|   |   |-- decimal-value.ts    exact financial value rules
|   |   |-- report-calculator.ts declarative total calculations
|   |-- application/
|   |   |-- funding-panel.store.ts
|   |   |-- funding-cell-editor.ts
|   |-- data-access/
|   |   |-- funding-panel.dto.ts
|   |   |-- funding-panel.schema.ts
|   |   |-- funding-report.mapper.ts
|   |   |-- funding-panel.gateway.ts
|   |   |-- http-funding-panel.gateway.ts
|   |   |-- mock-funding-panel.gateway.ts
|   |-- presentation/
|   |   |-- funding-grid.viewmodel.ts
|   |   |-- funding-grid/
|   |   |   |-- funding-grid.component.*
|   |   |   |-- funding-grid-display.ts
|   |   |   |-- funding-grid.theme.ts
|   |   |   |-- funding-amount-cell-editor.component.*
|   |-- panels/sec-corp/
|   |   |-- sec-corp-panel.component.*
|   |   |-- sec-corp-row-catalog.ts
|   |-- mocks/sec-corp-report.fixture.json
```

Why this split

- **Domain stays UI-free:** rows, periods, permissions, and calculations contain no grid classes or dirty flags.
- **View models stay transport-free:** cells add editable, dirty, preview, and validation state only after domain mapping.
- **DTOs are not domain objects:** Zod validates unknown JSON before mapping it into trusted domain types.
- **Future panels remain small:** add panel composition, row metadata, and backend data without cloning the store or grid.

Reuse boundary for future panels

A new funding panel should reuse `FundingPanelStore`, `FundingGridComponent`, the financial editor, the gateway token/providers, decimal utilities, host adapter, and DTO/domain separation. Panel-specific code should normally be limited to routing/composition, row definitions, mock fixtures, and genuinely different calculation rules.

4. Domain model and view-model mapping

Core domain model

Type	Important fields	Purpose
FundingReport	reportId, panelCode, businessDate, currency, timezone, asOf, version, permissions, periods, rows	Complete authoritative report.
ReportPeriod	id, label, kind, editable	Defines ordered snapshot/live/opportunity columns.
FundingRow	id, code, label, displayOrder, depth, kind, valueMode, calculation, values	Financial row independent of AG Grid.
FundingValueMap	Record	Exact values for all periods.
SaveFundingReportComm and	expectedVersion, businessDate, snapshotValues	Complete editable replacement sent by PUT.

Period contract

Order	Domain ID	Display	Behavior
1	snapshot0830	8:30	Editable on permitted input rows.
2	snapshot1130	11:30	Editable on permitted input rows; 0.00 when not available.
3	snapshot1330	1:30	Editable on permitted input rows; 0.00 when not available.
4	live	LIVE	Always read-only.
5	opportunityFunding	Opps funding	Always read-only.

Exact-value policy

No floating-point money

REST monetary values are signed decimal strings with exactly two fractional digits. Example: -308824714.48. Input normalization and calculation use decimal.js. A missing snapshot property is a contract failure; an unavailable snapshot value is explicitly 0.00.

Domain to grid view model

Domain input	View-model output
FundingReport metadata	Grid title, business date, currency, timezone, as-of text, and version.
FundingRow	FundingGridRowViewModel with the same semantic identity and order.
Row value + permissions	FundingGridCellViewModel value and editable flag.
Sparse committed edits	dirty flag and overlaid value.
Active editor	preview flag and validation message.
Row kind/depth	Centralized row/cell classes and formatted hierarchy label.

The mapping is one-way and computed. AG Grid uses readOnlyEdit and never mutates domain or view-model rows directly.

5. Signals state design and editing workflow

State representations

Signal/state	Meaning
serverReport	Latest complete report confirmed by GET or successful PUT.
edits	Sparse immutable overlay of committed values not yet accepted by the server.
activeEdit	Current raw input, normalized validation result, row ID, and snapshot ID.
report (computed)	Server report plus valid overlay and active preview, with recalculated totals.
viewModel (computed)	Grid-ready rows/cells with dirty, editable, preview, and validation flags.
load/save/conflict	Operational status exposed to the panel and enterprise shell.

Edit and autosave sequence

Step	Behavior
1	Single click, Enter, F2, or direct typing starts the custom financial editor.
2	Every keystroke is validated. Valid input updates computed totals immediately; invalid input remains local to the editor.
3	Focus loss, Enter, or Tab commits a valid value into the sparse edit overlay.
4	Commit immediately queues a full-state versioned PUT. There is no persistent Save button.
5	Escape cancels the preview. Invalid input blocks navigation until corrected or cancelled.
6	The successful server response becomes the new authoritative report and accepted dirty markers clear.

Serialized and coalesced saves

Only one PUT can be in flight per panel store. Edits made during that request remain in the overlay. When the response arrives, accepted values are removed, newer user intent is retained, and one next PUT sends the latest complete state against the newly confirmed version. This avoids parallel write races without losing rapid edits.

Effects policy

Effects are restricted to imperative boundaries: publishing panel state to the shell, reacting to the shell refresh revision, and synchronizing accessibility metadata through AG Grid's API. Financial derivation and view-model construction remain pure computed signals.

6. AG Grid implementation plan

Column and edit policy

- Six visible columns: Bucket, 8:30, 11:30, 1:30, LIVE, and Opps funding.
- Only input rows in the three snapshot columns can edit, and only when canEdit and canSave are both true.
- LIVE, Opps funding, section rows, subtotals, calculated rows, and closing balances remain read-only.
- Bucket receives more width; numeric columns retain minimum widths and use internal horizontal scrolling when docked narrowly.

Centralized presentation policy

Concern	Central implementation
Number formatting	Exact decimal grouping; negatives use accounting parentheses and pink/red emphasis; zero is 0.00.
Row hierarchy	Depth-driven label indentation and stable semantic row classes.
Conditional styling	Opening/section/closing teal bands, subtotal muted teal, alternating detail rows, dirty/preview/error markers.
Validation	Custom financial editor plus AG Grid invalidEditValueMode=block.
Accessibility	Instance-safe descriptions and editor error IDs; AG Grid render API receives label and described-by metadata.
Responsive layout	Container query wraps action overlays; minimum column widths preserve financial readability.

Module and license strategy

The Enterprise license key is injected at runtime and never committed. The grid registers only the modules Sec Corp uses: client-side row model, cell/row styling, custom and text editing, tooltips, and render API. This keeps the feature tree-shakeable while allowing future licensed Enterprise modules to be added deliberately.

Visual target

The panel follows the approved screenshots: compact 20-pixel rows, a 30-pixel header, dark charcoal detail rows, teal group and total bands, tabular right-aligned numbers, restrained grid lines, and accounting negative colors. The shell-owned title bar is intentionally absent from the Angular panel.

7. REST JSON contract and optimistic concurrency

Retrieve

GET /api/v1/funding-panels/sec-corp?businessDate=2026-07-25

Response:

```
{
  "schemaVersion": 1,
  "reportId": "sec-corp-2026-07-25",
  "panelCode": "sec-corp",
  "businessDate": "2026-07-25",
  "currency": "USD",
  "timezone": "America/New_York",
  "asOf": "2026-07-25T14:00:17-04:00",
  "version": 17,
  "permissions": { "canEdit": true, "canSave": true },
  "periods": [ ...exactly five ordered periods... ],
  "rows": [ ...complete ordered report... ]
}
```

Save

PUT /api/v1/funding-panels/sec-corp/sec-corp-2026-07-25
If-Match: "17"
Content-Type: application/json

```
{
  "schemaVersion": 1,
  "expectedVersion": 17,
  "businessDate": "2026-07-25",
  "snapshotValues": {
    "occ": {
      "snapshot0830": "-308824714.48",
      "snapshot1130": "-306000000.00",
      "snapshot1330": "-302500000.00"
    }
  }
}
```

The actual PUT contains every input row. It is a complete replacement of editable snapshot state, not a partial cell patch.

Atomic concurrency rule

Step	Client	Server
1	GET receives version 17.	Returns complete authoritative report.
2	User edits locally; totals preview.	Database remains unchanged.
3	PUT sends expectedVersion 17 and If-Match "17".	Atomically compares current version.
4A	Accepts complete response version 18.	Persists all snapshots, recalculates, increments version.
4B	On 409/412, retains dirty edits and blocks refresh.	Rejects the entire stale write; applies no partial update.

Conflict policy

The client never silently rebases and overwrites newer data. A conflict preserves local edits and offers a two-step destructive action: Discard my edits and reload. Non-conflict save failures preserve dirty work and offer Retry using the same expected version.

8. Sec Corp source mapping and representative data

Area 2 of the approved color-panel reference defines the target structure. Placeholder rows are resolved by position from the detailed source table in area 1. If a source value cannot be resolved reliably, it is omitted rather than invented. The implemented catalog resolves all six visible placeholders.

Target position	Resolved source label
Margin after Fails margin 116-02149	Margin Fails (090 Account) Intl & MtM
Margin after Margin 580-10500	Arb MtM Wires
Next margin row	SLAB MtM Wires
Wires after Cust Wires 177/Omni/177 Pay	ARB SNC
Wires after Sec Corp/ARB Margin	OMNI
Next wires row	Intraday change

Row model

- The representative Sec Corp fixture contains 37 ordered rows.
- Rows cover opening balance, margin, wires, reserve requirement, other, settlement activity, financing activity, and end of day.
- SOD Balance is normalized as the single target opening row.
- Section rows carry null cells; unavailable business snapshots carry explicit 0.00.
- The shared funding-panel contract does not force future panels to contain exactly 37 rows.

Calculation structure

Calculated rows declare sum dependencies by stable row ID. The calculation service applies them per period using exact decimal arithmetic. This keeps total definitions testable and independent of the grid row position while preserving the screenshot order through displayOrder.

9. Approved decision register - ten review questions

The following register records the final outcome of the ten-question architecture review. These are approved decisions, not pending recommendations.

#	Decision	Approved outcome and architectural consequence
1	Snapshot validity	8:30, 11:30, 1:30, and LIVE are all valid snapshots. None is treated as a defective duplicate. If 11:30 or 1:30 business data is not available, its cells contain explicit 0.00; the column remains present.
2	Opps funding	Keep Opps funding read-only. It is part of the fixed report contract and visible grid but is not editable in this phase.
3	Source placeholders	Resolve question-mark rows from the detailed source table by position. Use values from area 1 when reliably identifiable; skip an unresolved row rather than inventing data. All six visible placeholders were resolved.
4	Editability boundary	Only input rows in 8:30, 11:30, and 1:30 can edit. LIVE, Opps funding, section rows, calculated totals, subtotals, and closing balances remain read-only. Permissions must allow both edit and save.
5	Autosave and concurrency	Save when editing completes on focus loss, Enter, or Tab using REST PUT. PUT replaces the complete editable snapshot state and includes expectedVersion plus If-Match. A stale version is rejected atomically with 409 or 412.
6	Preview calculations	Recalculate totals before save. Every valid keystroke updates dependent totals so the user sees the effect before leaving the cell. Invalid raw input is retained by the editor but excluded from calculations and saving.
7	AG Grid licensing	The enterprise application has an AG Grid Enterprise license. Inject the key at runtime and never commit it. Register only required modules now; add licensed modules deliberately when future panels need them.
8	Refresh model	No polling. Refresh is initiated manually by the shell. A refresh during save is queued; invalid edits, failed saves, and conflicts block refresh to prevent data loss.
9	Locale metadata	Currency and timezone come from the API report. They are domain metadata, not hardcoded presentation constants. The shell/deployment supplies the active ISO business date at runtime.
10	Angular runtime model	Use Angular 21 standalone, signals-first, OnPush, and zoneless. The shell remains responsible for chrome, docking, and dimensions; the Angular panel is a focused, independently testable feature surface.

Most consequential decision

Version-based optimistic concurrency is mandatory because autosave can occur from multiple users and panels. The database compare-and-increment must be atomic; client checks alone are not sufficient.

10. Testing and quality strategy

Test pyramid

Level	Required coverage
Domain unit tests	Decimal normalization, dependency totals, row validation, missing values, and exact expected results.
Mapper/schema tests	Valid responses, malformed JSON, period ordering, permissions, canonical decimals, and version conflict payloads.
Store tests	Preview totals, invalid edit blocking, serialized/coalesced saves, reverts during save, load/save retry, conflict retention, and refresh coordination.
Component tests	37-row rendering, exact columns, editing boundary, editor accessibility/validation, Escape cancellation, and AG Grid API metadata.
Gateway tests	GET query, complete PUT body, If-Match, 409 and 412 translation, and mock version advancement.
Deployment smoke test	Real shell configuration, keyboard workflow, API version increments, stale-write rejection, responsive docking, and license activation.

Release gate

```
npm ci
npm run check
npm audit --omit=dev

npm run check =
  format:check
+ lint
+ 66 unit/component tests
+ production build
```

Current verified baseline

- Formatting and ESLint pass.
- 66 tests pass across 16 test files.
- Production build completes without warnings.
- Initial bundle: 230.88 kB raw, 64.42 kB estimated transfer.
- Sec Corp lazy AG Grid chunk: 1.19 MB raw, 255.75 kB estimated transfer.
- Production dependency audit: zero vulnerabilities.

A development-only transitive advisory remains on Angular CLI -> MCP SDK -> Hono. The forced fix would downgrade the Angular CLI and is not applied. It should be rechecked during routine dependency upgrades.

11. Delivery phases and implementation status

Phase	Scope	Status
1 - Foundation	Angular 21 strict standalone scaffold, feature boundaries, lint/format/test baseline.	Complete
2 - Contract	Domain model, decimal rules, REST DTO/schema/mapper, 37-row mock data.	Complete
3 - State	Signals store, computed preview/view model, serialized save queue, shell adapter.	Complete
4 - Grid	Read-only AG Grid, screenshot-aligned theme, centralized formatting.	Complete
5 - Editing	Custom editor, validation, dirty state, autosave, retry/conflict UI.	Complete
6 - Runtime	Mock/HTTP selection, business date injection, production host refresh.	Complete
7 - Hardening	Accessibility, responsive controls, error coverage, release gate, runbook.	Complete

Production handoff items

- Inject the real ISO business date and AG Grid Enterprise key before the Angular bundle starts.
- Set fundingPanelDataSource to http and supply the production API base URL.
- Verify backend atomic compare/update/version increment and authoritative recalculation.
- Run the deployment smoke test with real authenticated credentials and multiple concurrent users.
- Confirm API monitoring for latency, 5xx failures, and expected 409/412 conflict counts.

Readiness statement

The panel architecture and implementation are complete through Phase 7. Remaining items are environment integration and backend verification, not unresolved client architecture decisions.

12. Runtime configuration and operational behavior

Runtime configuration

```
window.__SEC_CORP_PANEL_CONFIG__ = {
  apiUrl: "/api",
  agGridEnterpriseLicenseKey: "<deployment-managed key>",
  businessDate: "2026-07-28",
  fundingPanelDataSource: "http"
};
```

Operator-visible states

State	Behavior
Loading failure	Show service error and Try again. The active query remains available.
Save failure	Keep dirty values and show Retry. Refresh remains blocked.
Version conflict	Keep local edits. Require two-step confirmation before discard and reload.
Invalid edit	Keep editor open with inline message; block save and refresh.
Refresh during save	Queue one refresh and start GET after the save queue becomes clean.
Duplicate refresh	Reject while a GET is already active.

Security and audit boundaries

- The shell and API gateway own authentication and authorization context.
- The API enforces edit/save permissions and version atomicity; client permissions are a UX boundary, not the security boundary.
- Financial payloads and the Enterprise license key must not be written to client logs.
- Request correlation, user identity, and persistence audit records belong at the API/service boundary.

Appendix A. Architecture references

The detailed implementation records are maintained with the panel source. Paths are relative to the sec-corp-panel repository.

Reference	Path
Domain and REST contract	docs/architecture/phase-2-domain-contract.md
Signals state design	docs/architecture/phase-3-signals-state.md
AG Grid presentation	docs/architecture/phase-4-read-only-grid.md
Editing and autosave	docs/architecture/phase-5-editing-autosave.md
Runtime integration	docs/architecture/phase-6-runtime-integration.md
Production hardening	docs/architecture/phase-7-production-hardening.md
Operations runbook	docs/operations/sec-corp-panel-runbook.md
Domain model	src/app/features/funding-panel/domain/funding-report.ts
Signals store	src/app/features/funding-panel/application/funding-panel.store.ts
Grid component	src/app/features/funding-panel/presentation/funding-grid/funding-grid.component.ts
HTTP gateway	src/app/features/funding-panel/data-access/http-funding-panel.gateway.ts

End of document